프로젝트 보고서(총 15점)

학번	3114	성명	조성현
프로젝트명	배고 — 위치 기반 식사 추천 서비스 (Web-view App)		
개발 기간	2026년 3월 6일 - 2026년 4월 25일 (총 8주)		
담당 역할	Frontend (UI/UX) — Figma 화면 설계, 디자인 시스템 수립, 공통 컴포넌트 개발, 반응형 레이아웃 구현		

1. 개요 및 목표 (1점)

가. 프로젝트 기획 의도

점심·저녁 식사 시 메뉴를 결정하지 못하는 상황은 직장인과 학생 모두가 일상적으로 경험하는 문제다. 기존 네이버 지도, 카카오맵 등의 서비스는 사용자가 검색어를 직접 입력하거나 카테고리를 선택해야 하는 능동적 탐색 방식을 전제로 한다. 이는 메뉴 결정 과정에서 오히려 선택지를 늘려 결정 피로도를 높이는 역효과를 낳는다.

본 프로젝트 **배고**는 HTML5 Geolocation API로 사용자 위치를 자동 감지하고, Naver 지역 검색 API를 프록시 서버로 경유하여 신뢰도 높은 식당 데이터를 즉시 추천하는 **수동적(passive) 추천 서비스**를 구현하는 것을 목표로 한다. 검색어 입력 없이 터치 한 번으로 메뉴를 결정할 수 있는 최소 기능 제품(MVP)을 구축하고, 이후 사용자 선택 이력 데이터를 기반으로 개인화 추천 엔진으로 고도화하는 것을 장기 목표로 설정하였다.

나. 최종 요구사항 및 기능 명세

기능	요구사항	상태
위치 자동 감지	Geolocation API로 위경도 수집, 권한 거부 시 기본 좌표(강남역) fallback	완료
식당 추천	카테고리(한/중/일/양식), 반경(500m/1km/2km) 필터 기반 추천 결과 제공	완료
프록시 서버	Naver API 키 보안을 위한 서버사이드 프록시, CORS 처리	완료
캐싱	Redis로 식당 목록 TTL 30분, 역지오코딩 TTL 24시간 캐싱	완료

딥링크 연동	식당 클릭 시 네이버 지도 앱 또는 모바일 웹으로 직접 이동	완료
즐겨찾기	식당 즐겨찾기 저장 및 목록 조회 (localStorage 기반 MVP)	완료

2. 기능 구현 및 결과 (4점)

가. 전체 시스템 흐름도

사용자 (모바일 브라우저)

↓ HTML5 Geolocation API (위경도 수집)

↓ Zustand 전역 상태 (필터: 카테고리, 반경)

↓ Axios 요청 → NestJS 프록시 서버 (BE)

↓ Redis 캐시 확인 → HIT: 즉시 응답 / MISS: Naver API 호출

↓ Naver Search Local API / Reverse Geocoding API

↓ 응답 가공 후 FE 반환

↓ React 컴포넌트 렌더링 (RestaurantCard, BottomSheet)

↓ 식당 클릭 → 네이버 지도 앱 딥링크 (nmap://) 또는 웹 fallback

[그림 1] 배고 서비스 전체 시스템 데이터 흐름도

나. 주요 기능 구현 내역

기능 1. 공통 UI 컴포넌트 시스템 구축

- 구현 내용: Figma 디자인 시스템(컬러 팔레트 #FF6B35, 타이포그래피, 8px 그리드)을 tailwind.config.js에 커스텀 토큰으로 등록하여 디자인-코드 간 일관성을 확보하였다. Button, RestaurantCard, Tag, BottomSheet, LoadingSpinner, BottomNav 등 재사용 가능한 공통 컴포넌트를 제작하였다.

```
// tailwind.config.js
module.exports = {
  content: ["./src/**/*.{js,jsx,ts,tsx}"],
  theme: {
    extend: {
      colors: {
        primary: "#FF6B6B",
        secondary: "#4ECDC4",
        background: "#F7F9FC",
      },
      height: { 'screen-dvh': '100dvh' }
    },
  },
  plugins: [],
}
```

[코드 1] 디자인 시스템 컬러 및 dvh 유틸리티가 등록된 Tailwind 설정 파일

▪ 결과 화면: 스플래시 → 위치 감지 로딩 → 추천 카드 화면이 디자인 시안과 동일한 컬러·간격으로 렌더링된다.

▪ 예외 상황 처리: API 응답 대기 중 RestaurantCardSkeleton이 표시되어 빈 화면 없이 자연스러운 로딩 경험을 제공한다.

기능 2. 모바일 반응형 레이아웃 및 Safe Area 처리

▪ 구현 내용: 모바일 브라우저에서 100vh 사용 시 하단 네비게이션 바에 UI가 가려지는 문제를 CSS dvh(Dynamic Viewport Height) 단위 도입으로 해결하였다. 또한 iPhone Safe Area(홈바 영역) 침범 문제는 viewport-fit=cover 메타 태그 추가 및 env(safe-area-inset-bottom) CSS 변수 적용으로 해결하였다.

▪ 결과 화면: iPhone 15 Safari, Galaxy S24 Chrome, iPad Air 등 다양한 기기에서 레이아웃 깨짐 없이 정상 동작한다.

▪ 예외 상황 처리: dvh 미지원 구형 브라우저에서는 100vh로 자동 fallback되도록 CSS 선언 순서를 조정하였다.

기능 3. framer-motion 기반 BottomSheet 컴포넌트

▪ 구현 내용: 필터 선택 및 식당 상세 정보 표시를 위한 BottomSheet를 framer-motion의 motion.div + useDragControls로 구현하였다. 아래로 100px 이상 드래그 시 시트가 닫히는 스와이프 제스처와 y축 슬라이드 + opacity 페이드 애니메이션을 적용하였다.

```
// src/components/BottomSheet.tsx
import { motion, useDragControls } from 'framer-motion';

export const BottomSheet = ({ isOpen, onClose, children }) => {
  const dragControls = useDragControls();
  return (
    <motion.div
      drag="y" dragControls={dragControls}
      dragConstraints={{ top: 0 }} dragElastic={0.2}
      onDragEnd={(_, info) => { if (info.offset.y > 100) onClose(); }}
      initial={{ y: '100%' }}
      animate={{ y: isOpen ? 0 : '100%' }}
      transition={{ type: 'spring', damping: 30, stiffness: 300 }}
      className="fixed bottom-0 left-0 right-0 bg-white rounded-t-3xl shadow-2xl
p-6 z-50"
    >
      {children}
    </motion.div>
  );
};
```

[코드 2] framer-motion 기반 스와이프 제스처 BottomSheet 컴포넌트 핵심 코드

▪ 예외 상황 처리: 스와이프 도중 터치를 놓았을 때 100px 미만이면 시트가 원위치로 복귀하도록 onDragEnd 임계값 판별 로직을 적용하였다.

기능 4. Skeleton UI 컴포넌트

▪ 구현 내용: API 로딩 중 빈 화면 대신 실제 레이아웃과 동일한 구조의 Skeleton 컴포넌트(RestaurantCardSkeleton, HeaderSkeleton)를 Tailwind animate-pulse로 구현하였다.

▪ 예외 상황 처리: 로딩 완료 후 실제 데이터로 자연스럽게 전환되며, 에러 발생 시 Skeleton 대신 에러 안내 화면이 표시된다.

3. 서비스 연동 및 데이터 처리 (2점)

가. 외부 서비스 연동 내역

서비스	용도	연동 방식
Naver Search Local API	위치 기반 식당 목록 조회	BE 프록시 서버 경유, Client-ID/Secret 서버사이드 .env 관리
Naver Reverse Geocoding API	위경도 → 행정동 주소 변환	BE 프록시 서버 경유, 결과 Redis 24시간 캐싱
Redis 7.x	API 응답 캐싱, Rate Limit 카운터	ioredis 클라이언트, Docker 컨테이너로 BE와 함께 구동
Naver 지도 딥링크	식당 상세 정보 및 길 찾기 연동	nmap:// 앱 스킴 + map.naver.com 웹 fallback
AWS EC2 (t3.micro)	백엔드 서버 프로덕션 배포	Docker Compose, GitHub Actions CI/CD 자동 배포

나. 데이터 처리 방법

FE에서 useGeolocation 훅이 위경도를 수집하면 Zustand store의 필터 상태(카테고리, 반경)와 함께 Axios 인스턴스를 통해 BE 프록시 서버로 전달된다. BE(NestJS)는 요청 수신 즉시 ValidationPipe로 파라미터를 검증하고, CacheService를 통해 Redis에서 캐시 키(lat.toFixed(3):lng.toFixed(3):category:radius)를 조회한다. 캐시 HIT 시 Redis 데이터를 즉시 반환하며, MISS 시 Naver Search Local API를 호출하여 결과를 Redis에 저장(TTL 30분) 후 반환한다. FE는 수신한 식당 목록을 RestaurantCard 컴포넌트에 바인딩하여 렌더링한다. 즐겨찾기 데이터는 MVP 단계에서 localStorage에 저장되며, 추후 BE /api/favorites 엔드포인트와 동기화할 예정이다.

4. 유지보수 계획 (2점)

가. 주요 핵심 코드 설명

```
// src/components/RestaurantCardSkeleton.tsx
export const RestaurantCardSkeleton = () => {
  return (
    <div className="bg-white rounded-2xl shadow-lg p-5 flex flex-col gap-3
animate-pulse">
      <div className="flex justify-between items-center">
        <div className="h-6 w-32 bg-gray-200 rounded-md" />
        <div className="h-5 w-12 bg-gray-200 rounded-full" />
      </div>
      <div className="flex gap-4 mt-2">
        <div className="h-4 w-16 bg-gray-200 rounded-md" />
        <div className="h-4 w-16 bg-gray-200 rounded-md" />
      </div>
    </div>
```

```
  );
};
```

[코드 3] animate-pulse를 활용한 RestaurantCardSkeleton — 실제 카드와 동일한 레이아웃 구조 유지

위 컴포넌트는 실제 RestaurantCard와 동일한 DOM 구조를 유지하면서 콘텐츠 자리에 회색 블록과 pulse 애니메이션을 적용한다. 데이터 로딩 완료 후 실제 카드로 교체될 때 레이아웃 변화(Cumulative Layout Shift)가 발생하지 않아 사용자 경험이 개선된다.

나. 유지보수 계획

항목	현황	개선 계획
크로스 브라우징	iPhone/Android/iPad 주요 기기 테스트 완료	컴포넌트 완성 직후 실기기 테스트 의무화 규칙 적용
햅틱 피드백	Android Chrome에서만 동작 (iOS 미지원)	navigator.vibrate 지원 여부 런타임 감지 후 Android 전용 조건부 처리
성능 최적화	미측정	Lighthouse 측정 후 이미지 lazy loading, WebP 변환 적용
BottomSheet	framer-motion 기반 완료	저사양 기기 렌더링 성능 측정 후 애니메이션 최적화
접근성	미적용	aria-label, role 속성 추가 및 키보드 탐색 지원

5. 결론 및 고찰 (4점)

가. 최종 시연 결과

UI/UX 담당으로서 기획 단계에서 정의한 6개 화면(스플래시, 위치 감지, 추천 카드, 필터, 즐겨찾기, 마이페이지) 모두를 Figma 시안에 준하여 구현하였다. Tailwind CSS 기반 공통 컴포넌트 시스템을 수립하여 디자인 수정 시 단일 config 파일 변경으로 전체 화면에 일괄 반영되는 구조를 확보하였다. framer-motion 도입으로 BottomSheet 스와이프 제스처와 화면 전환 애니메이션이 네이티브 앱 수준의 자연스러운 UX를 제공한다. iPhone 15 Safari, Galaxy S24 Chrome에서 Safe Area 처리 및 dvh 단위 적용을 통해 레이아웃 깨짐 없는 크로스 브라우징을 달성하였다.

나. 고찰

UI/UX 담당으로서 이번 프로젝트를 통해 디자인 시스템 선행 정의의 중요성을 실감하였다. 초반에 Figma 컴포넌트 구조를 일반 그룹(Group)으로 제작하다가 Atomic Design 방식으로 전면 재설계한 경험은, Master Component를 먼저 정의한 후 화면 구성에 착수하는 순서가 장기적으로 유지보수 비용을 크게 낮춘다는 것을 보여주었다.

모바일 웹 환경의 뷰포트 처리(100vh vs dvh)와 iOS Safe Area 이슈는 데스크톱 위주로 학습한 CSS 지식만으로는 예측하기 어려운 문제였다. 실기기 테스트를 개발 중간중간 병행하지 않고 막바지에 몰아서 진행한 탓에 수정 사항이 집중되었고, 이는 다른 작업 일정에도 영향을 미쳤다. 앞으로는 컴포넌트 단위 완성 직후 실기기 검증을 습관화하는 것이 필요하다.

심화 연구 방향으로는 첫째, 현재 CSS 기반 애니메이션의 성능을 Lighthouse와 Chrome DevTools Performance 탭으로 측정하고, 저사양 기기에서 프레임 드롭이 발생하는 컴포넌트를 선별하여 GPU 가속(will-change, transform)을 적용하는 최적화 작업을 진행할 수 있다. 둘째, 서비스를 PWA(Progressive Web App)로 전환하여 홈 화면 추가, 오프라인 Skeleton 제공, 푸시 알림을 지원하는 방향으로 사용자 재방문율을 높일 수 있다.